

КОНТРОЛЬНАЯ РАБОТА 1

по дисциплине «ПО объектов наземной инфраструктуры»

Специальность: 27.03.05 Инноватика, профиль «Инновационные технологии беспилотных систем»

Курс: 3, бакалавриат

Продолжительность: 180 минут (2 учебные пары)

Формат: практическая работа в Qt Creator с обязательным использованием Qt Designer

Общие требования ко всем вариантам

Параметр	Требование
Проект	Qt Widgets Application, компилятор MinGW или MSVC
Интерфейс	Обязательно собирается в Qt Designer (файл .ui)
Графика	QGraphicsScene + QGraphicsView для визуализации
Таймер	QTimer для цикла симуляции (интервал 20–100 мс)
Объём	Одно окно, 5–8 виджетов, 1 графическая сцена
Сдача	Рабочий проект + устный ответ на 3 контрольных вопроса

Критерий оценки	Минимальный порог сдачи
Запуск	Проект компилируется и запускается без ошибок (3 балла)
Интерфейс	Соответствует эскизу варианта, все виджеты размещены и названы корректно (5 баллов)
Теория	Правильный ответ минимум на 2 из 3 контрольных вопросов (6 баллов)
Дополнительно	За усложнение (второй график, звук, анимация) — повышение оценки на один балл (7 баллов)

Общие пояснения по всем вариантам:

Звёздочки `\*` в коде — это символ указателя в C++, он встречается в двух значениях:

1. При объявлении переменной — «это указатель»

```
QGraphicsScene *scene; // scene — указатель на объект QGraphicsScene
QTimer *timer;         // timer — указатель на объект QTimer
```

Звёздочка перед именем означает: переменная хранит не сам объект, а его адрес в памяти. В Qt почти все объекты создаются через `new` и хранятся как указатели — это особенность фреймворка.

2. При использовании — «разыменуй указатель»

```
QPainterPath *estPath = new QPainterPath();
// ...
scene->addPath(*estPath, QPen(Qt::blue, 2));
```

Здесь `*estPath` означает «возьми объект, на который указывает `estPath`, и передай его целиком». Метод `addPath` принимает значение, а не указатель — поэтому звёздочка нужна, чтобы «распаковать» указатель.

Представьте камеру хранения в аэропорту. Выдаётся номерок (указатель), а не сам чемодан (объект). Чтобы получить чемодан, нужно «разыменовать» номерок — поставить `*`. А стрелочка `->` — это «открой чемодан и вызови его метод». В Qt это встречается постоянно, потому что `QGraphicsScene`, `QTimer`, `QGraphicsEllipseItem` и другие объекты всегда создаются динамически (через `new`) и передаются как указатели.

Qt Designer — это встроенный визуальный редактор интерфейсов, он не требует отдельной «подключки» в привычном смысле. Он уже идёт в комплекте с Qt Creator и активируется автоматически при открытии файла `.ui`. Как это работает на практике.

Способ 1. Автоматический (стандартный)

При создании проекта Qt Widgets Application Qt Creator сам создаёт файл `mainwindow.ui`. Двойной клик по нему — и открывается Qt Designer прямо внутри Qt Creator.

В дереве файлов слева найдите файл `mainwindow.ui`.

Дважды кликните по нему.

Откроется визуальный редактор: слева — панель виджетов (Widget Box), по центру — форма окна, справа — свойства (Property Editor).

Способ 2. Ручное добавление `.ui`-файла

Если вы создали проект без формы (галочка «Generate form» не стояла):

В дереве проекта кликните правой кнопкой мыши по папке проекта.

Add New... → Qt → Qt Designer Form Class.

Выберите шаблон (обычно Main Window).

Задайте имя класса (например, MainWindow).

Нажмите Finish — появится новый `.ui`-файл.

Частые проблемы

«Открывается текст XML вместо визуального редактора»

Файл `.ui` по ошибке открыт как обычный текст. Кликните по нему правой кнопкой и выберите Open With... → Qt Designer.

«Qt Designer вообще не установлен»

При установке Qt через Qt Maintenance Tool или Online Installer нужно было отметить галочку напротив компонента Qt Designer. Если его нет — откройте Qt Maintenance Tool, найдите раздел Qt версии, и поставьте галочку.

«Виджеты не перетаскиваются на форму»

Перетащите виджет из панели Widget Box (слева) прямо на серое окно формы. Если форма не видна — нажмите кнопку Edit в нижней панели Qt Creator.

Проверка, что всё работает

Признак	Что должно происходить
Двойной клик по .ui	Открывается визуальный редактор с формой
Слева — панель Widget Box	Видны категории: Layouts, Buttons, Input Widgets
Перетаскивание кнопки	Кнопка появляется на форме
Справа — Property Editor	При выделении виджета показываются его свойства
Кнопка Edit/Design	Переключает между кодом и визуальным редактором

Подключить Qt Designer = **открыть файл .ui** в Qt Creator. Отдельной настройки не требуется.

Сводная таблица вариантов

№	Тема	Ключевые виджеты	Ключевой алгоритм
1	Инерциальная навигация	LineEdit, Label, GraphicsView	Двойное интегрирование, дрейф
2	Сопровождение цели	Label, GraphicsView	Вычисление ошибки, движение к цели
3	Обход зоны РЭБ	LineEdit, GraphicsView	Пересечение отрезка и круга, arcTo
4	Конечный автомат	Button, Label, TextEdit, GraphicsView	switch по enum, приоритет состояний
5	Распределение в рое	Button, TableWidget, GraphicsView	Жадный аукционный алгоритм
6	Помехоустойчивое кодирование	LineEdit, Label, TextEdit	CRC16, инверсия бита
7	ПИД-стабилизация	Slider, GraphicsView ×2	ПИД-формула, накопление ошибки
8	Автовозврат (RTL)	Button, Label, TextEdit, GraphicsView	Маршрут + переход в режим RTL
9	Зона покрытия	Slider, GraphicsView	Расстояние точка-точка, клик по сцене
10	Мониторинг парка	TableWidget, GraphicsView, Timer	Периодическое обновление, подсветка строк

Вариант 1. Инерциальная навигация при потере GPS

Задача: Смоделировать движение БПЛА по инерциальным данным. Пользователь задаёт начальное ускорение по X и Y. На сцене отображается траектория. При нажатии кнопки «Коррекция» позиция сбрасывается к истинной (имитация внешней коррекции).

Эскиз окна:

Инерциальная навигация	
QGraphicsView (траектория) синяя – оценка зелёная – истина	Ускорение X: [0.5]
	Ускорение Y: [0.3]
	Дрейф: 0.02 м/с
	[Старт] [Коррекция]
	[Сброс]
	Статус: ...
	Ошибка: ... м

Виджеты Qt Designer:

objectName	Тип	Назначение
graphicsView	QGraphicsView	Отрисовка траекторий
lineEditAx	QLineEdit	Ввод ускорения по X
lineEditAy	QLineEdit	Ввод ускорения по Y
btnStart	QPushButton	Запуск симуляции
btnCorrect	QPushButton	Коррекция позиции
btnReset	QPushButton	Сброс
labelStatus	QLabel	Статус и величина ошибки

Алгоритм решения:

1. Qt Designer: Разместить виджеты по эскизу. graphicsView — слева, остальные — справа в вертикальном layout.

2. mainwindow.h — переменные:

```

QGraphicsScene *scene;
QTimer *timer;
double xEst, yEst, vx, vy; // оценка (с дрейфом)
double xTrue, yTrue;      // истинная позиция
double ax, ay;           // ускорения
double drift;             // коэффициент дрейфа
QPainterPath *estPath;    // след оценки
QPainterPath *truePath;   // след истины
QGraphicsPathItem *estItem, *trueItem;
```

3. Конструктор:

```
scene = new QGraphicsScene(0, 0, 600, 400, this);
ui->graphicsView->setScene(scene);
estItem = scene->addPath(*estPath, QPen(Qt::blue, 2));
trueItem = scene->addPath(*truePath, QPen(Qt::green, 2));
xEst = xTrue = 300; yEst = yTrue = 200;
timer = new QTimer(this);
connect(timer, &QTimer::timeout, this, &MainWindow::tick);
```

4. Слот tick() (вызывается каждые 50 мс, dt = 0.05):

```
// Истинная траектория (без шума)
xTrue += ax * 0.05 * 0.05 / 2;
yTrue += ay * 0.05 * 0.05 / 2;
// Оценка с дрейфом (имитация накопления ошибки ИНС)
vx += (ax + drift) * 0.05;
vy += (ay + drift) * 0.05;
xEst += vx * 0.05;
yEst += vy * 0.05;
// Обновление линий
estPath->lineTo(xEst, yEst);
truePath->lineTo(xTrue, yTrue);
estItem->setPath(*estPath);
trueItem->setPath(*truePath);
// Вывод ошибки
double err = std::sqrt(std::pow(xEst - xTrue, 2) + std::pow(yEst - yTrue, 2));
ui->labelStatus->setText(QString("Ошибка: %1 м").arg(err, 0, 'f', 1));
```

5. Кнопка «Коррекция»:

```
xEst = xTrue; yEst = yTrue;
vx = 0; vy = 0;
```

Контрольные вопросы:

№	Вопрос
1	Почему инерциальная навигация накапливает ошибку со временем? Назовите физическую причину.
2	Что такое комплементарный фильтр и как он объединяет данные ИНС и внешних датчиков?
3	Как часто требуется коррекция ИНС для БПЛА малого класса при типовом дрейфе 0,01 м/с?

Вариант 2. Обнаружение и сопровождение объекта на видеопотоке

Задача: На графической сцене движется «цель» (красный прямоугольник случайного размера). БПЛА (синий круг в центре) должен вычислять смещение центра цели относительно себя и выводить ошибки по X и Y. При нажатии «Сопровождение» БПЛА начинает двигаться к центру цели.

Эскиз окна:

Сопровождение цели	
QGraphicsView	Ошибка X: +45
красный квадрат – цель	Ошибка Y: -20
синий круг – БПЛА	[Захват цели]
зелёная линия – линия визирования	[Сопровождение]
	[Сброс]
	Режим: Ожидание

Виджеты Qt Designer:

objectName	Тип	Назначение
graphicsView	QGraphicsView	Сцена с объектами
btnCapture	QPushButton	Захват цели
btnTrack	QPushButton	Включение сопровождения
btnReset	QPushButton	Сброс
labelErrorX	QLabel	Ошибка по X
labelErrorY	QLabel	Ошибка по Y
labelMode	QLabel	Текущий режим

Алгоритм решения:

1. Qt Designer: Разместить виджеты по эскизу.

2. mainwindow.h:

```
QGraphicsScene *scene;
QGraphicsEllipseItem *uav;    // БПЛА
QGraphicsRectItem *target;    // цель
QGraphicsLineItem *line;      // линия визирования
QTimer *timer;
double targetX, targetY;      // координаты цели
double uavX, uavY;            // координаты БПЛА
bool tracking;                 // режим сопровождения
```

3. Конструктор:

```
scene = new QGraphicsScene(0, 0, 600, 400, this);
ui->graphicsView->setScene(scene);
uav = scene->addEllipse(290, 190, 20, 20, QPen(Qt::blue), QBrush(Qt::blue));
targetX = 100 + rand() % 400;
targetY = 50 + rand() % 300;
target = scene->addRect(targetX, targetY, 30, 30, QPen(Qt::red), QBrush(Qt::red));
```

```

uavX = 300; uavY = 200;
tracking = false;
timer = new QTimer(this);
connect(timer, &QTimer::timeout, this, &MainWindow::tick);

```

4. Слот tick() (50 мс):

```

// Цель медленно дрейфует
targetX += (rand() % 5 - 2);
targetY += (rand() % 5 - 2);
target->setPos(targetX, targetY);
// Вычисление ошибки
double errX = targetX + 15 - uavX;
double errY = targetY + 15 - uavY;
ui->labelErrorX->setText(QString("Ошибка X: %1").arg(errX, 0, 'f', 0));
ui->labelErrorY->setText(QString("Ошибка Y: %1").arg(errY, 0, 'f', 0));
// Линия визирования
line->setLine(uavX, uavY, targetX + 15, targetY + 15);
// Сопровождение: БПЛА движется к цели
if (tracking) {
    double dist = std::sqrt(errX*errX + errY*errY);
    if (dist > 5) {
        uavX += (errX / dist) * 2;
        uavY += (errY / dist) * 2;
        uav->setPos(uavX - 10, uavY - 10);
    }
}

```

5. Кнопка «Сопровождение»: tracking = true; timer->start(50);

6. Кнопка «Захват цели»: перерисовать цель в новой случайной позиции, tracking = false.

Контрольные вопросы:

№	Вопрос
1	Чем отличается метод фоновой вычитания от контурного анализа при обнаружении объектов?
2	Как ПИД-регулятор используется для удержания цели в центре кадра? Какие ошибки подаются на его вход?
3	Что такое «потеря слежения» и какие программные стратегии используются для её обработки?

Вариант 3. Маршрутизация с обходом зоны РЭБ

Задача: На сцене отображаются старт (зелёный квадрат), цель (красный квадрат) и зона РЭБ (жёлтый круг). По кнопке «Проложить маршрут» строится путь из стартовой точки к цели, обходящий зону РЭБ. Путь отображается ломаной линией.

Эскиз окна:

Обход зоны РЭБ	
QGraphicsView	Старт X: [50] Старт Y: [350] Цель X: [550] Цель Y: [50] Радиус зоны: [120]
жёлтый круг – зона РЭБ	
зелёный – старт	
красный – цель	[Проложить маршрут]
синяя линия – путь	[Сброс] Длина пути: ...

Виджеты Qt Designer:

objectName	Тип	Назначение
graphicsView	QGraphicsView	Карта с зоной и маршрутом
lineEditStartX	QLineEdit	Координаты старта X
lineEditStartY	QLineEdit	Координаты старта Y
lineEditGoalX	QLineEdit	Координаты цели X
lineEditGoalY	QLineEdit	Координаты цели Y
lineEditRadius	QLineEdit	Радиус зоны РЭБ
btnRoute	QPushButton	Расчёт маршрута
btnReset	QPushButton	Очистка сцены
labelLength	QLabel	Длина пути

Алгоритм решения:

1. Qt Designer: Разместить виджеты по эскизу.

2. mainwindow.h:

```
QGraphicsScene *scene;
QGraphicsEllipseItem *zone;    // зона РЭБ
QGraphicsRectItem *start, *goal;
QGraphicsPathItem *pathItem;
double zoneX, zoneY, zoneR;
```

3. Конструктор: создать сцену, нарисовать зону РЭБ (круг), старт и цель.

4. Кнопка «Проложить маршрут» — упрощённый алгоритм обхода:

```
// Чтение параметров
double sx = ui->lineEditStartX->text().toDouble();
double sy = ui->lineEditStartY->text().toDouble();
double gx = ui->lineEditGoalX->text().toDouble();
```

```

double gy = ui->lineEditGoalY->text().toDouble();
double r = ui->lineEditRadius->text().toDouble();

QPainterPath path;
path.moveTo(sx, sy);
// Проверка: пересекает ли отрезок круг
double dx = gx - sx, dy = gy - sy;
double a = dx*dx + dy*dy;
double b = 2 * (sx*dx + sy*dy - zoneX*dx - zoneY*dy);
double c = (sx-zoneX)*(sx-zoneX) + (sy-zoneY)*(sy-zoneY) - r*r;
double disc = b*b - 4*a*c;
if (disc < 0) {
    // Нет пересечения — прямая
    path.lineTo(gx, gy);
} else {
    // Есть пересечение — обход по двум точкам на окружности
    double ang_start = std::atan2(sy - zoneY, sx - zoneX);
    double ang_goal = std::atan2(gy - zoneY, gx - zoneX);
    double ax = zoneX + r * std::cos(ang_start);
    double ay = zoneY + r * std::sin(ang_start);

    path.lineTo(ax, ay);
    path.arcTo(zoneX - r, zoneY - r, 2*r, 2*r,
               ang_start * 180/M_PI,
               (ang_goal - ang_start) * 180/M_PI);
    path.lineTo(gx, gy);
}

scene->addPath(path, QPen(Qt::blue, 2));
double len = path.length();
ui->labelLength->setText(QString("Длина пути: %1").arg(len, 0, 'f', 0));

```

5. Кнопка «Сброс»: очистить сцену, перерисовать зону, старт, цель.

Контрольные вопросы:

№	Вопрос
1	В чём суть алгоритма A* и почему он предпочтительнее поиска в ширину для маршрутизации БПЛА?
2	Как задаётся стоимость ребра в графе маршрутизации при наличии зон РЭБ?
3	Что такое адаптивная дискретизация местности и как она снижает вычислительную сложность?

Вариант 4. Конечный автомат отказоустойчивости

Задача: Реализовать конечный автомат с 4 состояниями: «Полёт», «Потеря связи», «Низкий заряд», «Посадка». Пользователь кнопками имитирует события (потеря связи, разряд батареи). БПЛА (круг на сцене) реагирует по состоянию: зелёный — полёт, жёлтый — потеря связи (возврат), оранжевый — низкий заряд, красный — посадка.

Эскиз окна:

Конечный автомат отказоустойчивости	
QGraphicsView круг – БПЛА зелёный – база линия – путь к базе	Состояние: ПОЛЁТ Заряд: 80% Связь: есть [Потеря связи] [Разряд батареи] [Восстановить] [Сброс] Лог событий: (QTextEdit)

Виджеты Qt Designer:

objectName	Тип	Назначение
graphicsView	QGraphicsView	Сцена с БПЛА и базой
btnLostLink	QPushButton	Имитация потери связи
btnLowBattery	QPushButton	Имитация разряда
btnRestore	QPushButton	Восстановление связи
btnReset	QPushButton	Полный сброс
labelState	QLabel	Текущее состояние
labelBattery	QLabel	Уровень заряда
textLog	QTextEdit	Лог событий

Алгоритм решения:

1. Qt Designer: разместить виджеты, textLog сделать readOnly.

2. mainwindow.h:

```
enum class State { FLIGHT, LOST_LINK, LOW_BATTERY, LANDING };
State currentState;
QGraphicsScene *scene;
QGraphicsEllipseItem *uav;
QTimer *timer;
int battery;
bool linkActive;
double uavX, uavY;
double baseX, baseY;
```

3. Конструктор:

```
scene = new QGraphicsScene(0, 0, 600, 400, this);
ui->graphicsView->setScene(scene);
```

```

scene->addRect(50, 350, 30, 30, QPen(Qt::green), QBrush(Qt::green));
uav = scene->addEllipse(290, 190, 20, 20);
uav->setBrush(Qt::green);
baseX = 65; baseY = 365;
uavX = 300; uavY = 200;
battery = 80;
linkActive = true;
currentState = State::FLIGHT;
timer = new QTimer(this);
connect(timer, &QTimer::timeout, this, &MainWindow::tick);
timer->start(50);

```

4. Слот `tick()` — конечный автомат:

```

switch (currentState) {
    case State::FLIGHT:
        uav->setBrush(Qt::green);
        ui->labelState->setText("Состояние: ПОЛЁТ");
        uavX += (rand() % 5 - 2);
        uavY += (rand() % 5 - 2);
        break;
    case State::LOST_LINK:
        uav->setBrush(Qt::yellow);
        ui->labelState->setText("Состояние: ВОЗВРАТ НА БАЗУ");
        moveTowards(baseX, baseY);
        if (distance(uavX, uavY, baseX, baseY) < 5) {
            log("На базе");
            currentState = State::LANDING;
        }
        break;
    case State::LOW_BATTERY:
        uav->setBrush(QColor(255, 165, 0));
        ui->labelState->setText("Состояние: НИЗКИЙ ЗАРЯД");
        moveTowards(baseX, baseY);
        break;
    case State::LANDING:
        uav->setBrush(Qt::red);
        ui->labelState->setText("Состояние: ПОСАДКА");
        break;
}
battery -= 1;
if (battery <= 20 && currentState == State::FLIGHT) {
    currentState = State::LOW_BATTERY;
    log("Низкий заряд!");
}
ui->labelBattery->setText(QString("Заряд: %1%").arg(battery));
uav->setPos(uavX - 10, uavY - 10);

```

5. Кнопки:

// Потеря связи

```
void on_btnLostLink_clicked() {
    linkActive = false;
    if (currentState == State::FLIGHT) {
        currentState = State::LOST_LINK;
        log("Потеря связи!");
    }
}
```

// Восстановить

```
void on_btnRestore_clicked() {
    linkActive = true;
    if (currentState == State::LOST_LINK) {
        currentState = State::FLIGHT;
        log("Связь восстановлена");
    }
}
```

Контрольные вопросы:

№	Вопрос
1	Что такое конечный автомат (State Machine) и почему он применяется в системах отказоустойчивости БПЛА?
2	Как приоритизируются аварийные состояния при одновременной потере связи и разряде батареи?
3	Что такое режим RTL (Return-to-Launch) и какие параметры полёта он использует?

Вариант 5. Распределение задач в рое (аукционный метод)

Задача: На сцене 3 БПЛА (круги разного цвета) и 3 зоны наблюдения (пунктирные квадраты). По кнопке «Распределить» каждому БПЛА назначается ближайшая зона. После назначения каждый БПЛА летит к своей зоне. Расчёт и отображение результата — в таблице.

Эскиз окна:

Распределение задач в рое	
QGraphicsView	[Распределить] [Сброс]
3 круга – БПЛА	Таблица:
3 квадрата – зоны	БПЛА Зона S
линии – назначения	1 3 120
	2 1 80
	3 2 200
	Сумма: 400

Виджеты Qt Designer:

objectName	Тип	Назначение
graphicsView	QGraphicsView	Сцена с дронами и зонами
btnAssign	QPushButton	Распределение зон
btnReset	QPushButton	Сброс позиций
tableWidget	QTableWidget	Таблица назначений
labelTotal	QLabel	Суммарная стоимость

Алгоритм решения:

1. Qt Designer: tableWidget — 3 колонки: «БПЛА», «Зона», «Расстояние».

2. mainwindow.h:

```
struct Drone { QPointF pos; QColor color; QGraphicsEllipseItem *item; int assignedZone; };
struct Zone { QPointF pos; QGraphicsRectItem *item; };
QList<Drone> drones;
QList<Zone> zones;
QTimer *timer;
```

3. Конструктор:

```
// 3 БПЛА в разных углах
drones << Drone{{100, 100}, Qt::red, scene->addEllipse(90, 90, 20, 20), -1};
drones << Drone{{500, 100}, Qt::blue, scene->addEllipse(490, 90, 20, 20), -1};
drones << Drone{{300, 350}, Qt::magenta, scene->addEllipse(290, 340, 20, 20), -1};
for (auto &d : drones) d.item->setBrush(d.color);

// 3 зоны
zones << Zone{{200, 200}, scene->addRect(190, 190, 40, 40, QPen(Qt::DashLine))};
zones << Zone{{450, 250}, scene->addRect(440, 240, 40, 40, QPen(Qt::DashLine))};
```

```
zones << Zone{{150, 320}, scene->addRect(140, 310, 40, 40, QPen(Qt::DashLine))};
```

4. Кнопка «Распределить» — жадный аукционный алгоритм:

```
QSet<int> taken;
for (int i = 0; i < drones.size(); ++i) {
    double bestDist = 1e9;
    int bestZone = -1;
    for (int j = 0; j < zones.size(); ++j) {
        if (taken.contains(j)) continue;
        double d = QLineF(drones[i].pos, zones[j].pos).length();
        if (d < bestDist) { bestDist = d; bestZone = j; }
    }
    drones[i].assignedZone = bestZone;
    taken.insert(bestZone);

    scene->addLine(drones[i].pos.x(), drones[i].pos.y(),
                  zones[bestZone].pos.x(), zones[bestZone].pos.y(),
                  QPen(drones[i].color, 2));

    ui->tableWidget->setItem(i, 0, new QTableWidgetItem(QString::number(i+1)));
    ui->tableWidget->setItem(i, 1, new QTableWidgetItem(QString::number(bestZone+1)));
    ui->tableWidget->setItem(i, 2, new QTableWidgetItem(QString::number(bestDist, 'f',
0)));
}
timer->start(30);
```

5. Слот tick() — движение к назначенным зонам:

```
for (auto &d : drones) {
    if (d.assignedZone < 0) continue;
    QPointF target = zones[d.assignedZone].pos;
    QPointF dir = target - d.pos;
    double dist = dir.length();
    if (dist > 3) {
        d.pos += dir / dist * 2;
        d.item->setPos(d.pos.x() - 10, d.pos.y() - 10);
    }
}
```

Контрольные вопросы:

№	Вопрос
1	В чём суть аукционного алгоритма распределения задач? Чем он отличается от централизованного планирования?
2	Что произойдёт, если число БПЛА меньше числа зон? Как модифицировать алгоритм?
3	Какие протоколы связи используются для координации роя (mesh, широковещание)?

Вариант 6. Помехоустойчивое кодирование команд (CRC)

Задача: Пользователь вводит текстовую команду (например, «FLY 100 200»). Программа формирует пакет: команда + CRC16. В поле «Принятый пакет» отображается результат. Кнопка «Имитация помехи» инвертирует один случайный байт. Кнопка «Проверить» определяет, прошёл пакет проверку CRC или повреждён.

Эскиз окна:

Помехоустойчивое кодирование
Команда: [FLY 100 200_____]
[Сформировать пакет] [Имитация помехи] [Проверить]
Исходный пакет: 46 4C 59 ... CRC=2A1F Принятый пакет: 46 4C 59 ... CRC=2A1F Результат: ОК / ОШИБКА CRC
Лог передач (QTextEdit)

Виджеты Qt Designer:

objectName	Тип	Назначение
lineEditCmd	QLineEdit	Ввод команды
btnSend	QPushButton	Формирование пакета
btnNoise	QPushButton	Имитация помехи
btnCheck	QPushButton	Проверка CRC
labelOriginal	QLabel	Исходный пакет
labelReceived	QLabel	Принятый пакет (после помехи)
labelResult	QLabel	Результат проверки
textLog	QTextEdit	Лог передач

Алгоритм решения:

1. mainwindow.h:

```
QByteArray originalPacket;
QByteArray receivedPacket;
```

```
uint16_t crc16(const QByteArray &data) {
    uint16_t crc = 0xFFFF;
    for (int i = 0; i < data.size(); ++i) {
        crc ^= (uint8_t)data[i];
        for (int j = 0; j < 8; ++j) {
```

```

        if (crc & 1) crc = (crc >> 1) ^ 0xA001;
        else crc >>= 1;
    }
}
return crc;
}

```

2. Кнопка «Сформировать пакет»:

```

QByteArray cmd = ui->lineEditCmd->text().toUtf8();
uint16_t crc = crc16(cmd);
originalPacket = cmd;
originalPacket.append((crc & 0xFF));
originalPacket.append((crc >> 8) & 0xFF);
receivedPacket = originalPacket;
ui->labelOriginal->setText(packetToHex(originalPacket));
ui->labelReceived->setText(packetToHex(receivedPacket));
log("Пакет сформирован, CRC = 0x" + QString::number(crc, 16).toUpper());

```

3. Кнопка «Имитация помехи»:

```

if (receivedPacket.size() < 3) return;
int pos = rand() % (receivedPacket.size() - 2);
receivedPacket[pos] ^= (1 << (rand() % 8));
ui->labelReceived->setText(packetToHex(receivedPacket));
log("Помеха: инвертирован бит в байте " + QString::number(pos));

```

4. Кнопка «Проверить»:

```

if (receivedPacket.size() < 3) return;
QByteArray data = receivedPacket.left(receivedPacket.size() - 2);
uint16_t receivedCrc = (uint8_t)receivedPacket[receivedPacket.size()-2]
    | ((uint8_t)receivedPacket[receivedPacket.size()-1] << 8);
uint16_t computedCrc = crc16(data);
if (receivedCrc == computedCrc) {
    ui->labelResult->setText("Результат: ОК — пакет цел");
    log("Проверка: ОК");
} else {
    ui->labelResult->setText("Результат: ОШИБКА CRC — пакет повреждён");
    log("Проверка: ОШИБКА CRC");
}

```

Контрольные вопросы:

№	Вопрос
1	Чем отличается обнаружение ошибок (CRC) от исправления ошибок (коды Рида-Соломона)?
2	Что такое ARQ (Automatic Repeat reQuest) и как он работает в связных каналах БПЛА?
3	Какой размер CRC16 достаточно для обнаружения двойных ошибок в пакете из 64 байт?

Вариант 7. Симулятор ПИД-стабилизации крена

ПИД-регулятор (пропорционально-интегрально-дифференциальный регулятор) — это алгоритм автоматического управления, который непрерывно вычисляет разницу (ошибку) между желаемым значением параметра (уставкой) и его текущим измеренным значением, а затем формирует управляющий сигнал для минимизации этой ошибки.

Расшифровка аббревиатуры

П (пропорциональная составляющая): реакция прямо пропорциональна текущей ошибке. Если отклонение большое — воздействие сильное, если маленькое — слабое.

И (интегральная составляющая): накапливает прошлые ошибки во времени. Нужна для устранения статической ошибки (когда система «зависает» чуть ниже или выше нужной точки).

Д (дифференциальная составляющая): реагирует на скорость изменения ошибки. Позволяет предсказать будущее отклонение и «гасить» колебания, делая систему устойчивее.

Задача: БПЛА (прямоугольник на сцене) наклоняется под действием случайного «ветра». Пользователь настраивает коэффициенты K_p , K_i , K_d тремя ползунками. ПИД-регулятор возвращает БПЛА к нулевому крену. График ошибки строится в реальном времени.

Эскиз окна:

ПИД-стабилизация крена	
QGraphicsView прямоугольник – БПЛА (наклон) вертикальная линия – целевой крен (0°)	Kp: —[====]— 2.0
	Ki: —[==]— 0.1
	Kd: —[====]— 0.5
	[Старт] [Сброс]
QGraphicsView (график ошибки)	Крен: 5.2°
	Ошибка: 5.2°
	Ветер: 3.1 м/с

Виджеты Qt Designer:

objectName	Тип	Назначение
graphicsView	QGraphicsView	Визуализация наклона БПЛА
graphicsViewChart	QGraphicsView	График ошибки
sliderKp	QSlider	Коэффициент пропорциональной составляющей
sliderKi	QSlider	Коэффициент интегральной составляющей
sliderKd	QSlider	Коэффициент дифференциальной составляющей
btnStart	QPushButton	Запуск
btnReset	QPushButton	Сброс
labelRoll	QLabel	Текущий крен
labelWind	QLabel	Текущий ветер

Алгоритм решения:**1. mainwindow.h:**

```

QGraphicsScene *scene, *chartScene;
QGraphicsRectItem *uav;
QTimer *timer;
double roll;
double integral, prevError;
double Kp, Ki, Kd;
double wind;
QList<double> errorHistory;

```

2. Конструктор:

```

scene = new QGraphicsScene(0, 0, 300, 300, this);
ui->graphicsView->setScene(scene);
uav = scene->addRect(-40, -10, 80, 20);
uav->setBrush(Qt::blue);
uav->setPos(150, 150);
scene->addLine(150, 50, 150, 250, QPen(Qt::green, 1, Qt::DashLine));

```

```

chartScene = new QGraphicsScene(0, 0, 300, 150, this);
ui->graphicsViewChart->setScene(chartScene);

```

```

roll = 0; integral = 0; prevError = 0;
Kp = 2.0; Ki = 0.1; Kd = 0.5;

```

3. Слот tick() (50 мс, dt = 0.05):

```

// Случайный ветер
wind += (rand() % 100 - 50) / 50.0;
wind = qBound(-5.0, wind, 5.0);

```

```

// ПИД
double error = 0.0 - roll;
integral += error * 0.05;
double deriv = (error - prevError) / 0.05;
double output = Kp * error + Ki * integral + Kd * deriv;
prevError = error;

// Применяем: выход регулятора компенсирует ветер
roll += (wind - output) * 0.05;
roll = qBound(-30.0, roll, 30.0);

// Визуализация наклона
uav->setRotation(roll);

// График
errorHistory.append(error);
if (errorHistory.size() > 200) errorHistory.removeFirst();
chartScene->clear();
for (int i = 1; i < errorHistory.size(); ++i) {
    chartScene->addLine(i-1, 75 - errorHistory[i-1]*2,
                       i, 75 - errorHistory[i]*2,
                       QPen(Qt::red, 1));
}

ui->labelRoll->setText(QString("Крен: %1°").arg(roll, 0, 'f', 1));
ui->labelWind->setText(QString("Ветер: %1 м/с").arg(wind, 0, 'f', 1));

```

Контрольные вопросы:

№	Вопрос
1	Как каждая из составляющих ПИД (П, И, Д) влияет на переходный процесс?
2	Что такое «насыщение интегратора» (integral windup) и как с ним бороться?
3	Почему дифференциальная составляющая чувствительна к шуму датчика и как это компенсировать?

Вариант 8. Имитация потери связи и автовозврата (RTL)

Задача: БПЛА летит по маршруту из 4 точек. Кнопка «Обрыв связи» запускает режим RTL: БПЛА летит к точке базы и «садится». В процессе на сцене меняется цвет БПЛА (зелёный → жёлтый → красный), а в логе фиксируются события.

Эскиз окна:

Автовозврат (RTL)	
QGraphicsView	[Старт миссии]
4 точки – маршрут	[Обрыв связи]
зелёный квадрат – база	[Сброс]
круг – БПЛА	Статус: ПОЛЁТ
пунктир – путь RTL	Связь: есть
	Лог (QTextEdit)

Виджеты Qt Designer:

objectName	Тип	Назначение
graphicsView	QGraphicsView	Сцена с маршрутом
btnStart	QPushButton	Запуск миссии
btnLostLink	QPushButton	Обрыв связи
btnReset	QPushButton	Сброс
labelStatus	QLabel	Статус
textLog	QTextEdit	Лог событий

Алгоритм решения:**1. mainwindow.h:**

```
enum Mode { MISSION, RTL, LANDED };
Mode mode;
QList<QPointF> waypoints;
int wpIndex;
QPointF base;
QGraphicsEllipseItem *uav;
QTimer *timer;
```

2. Конструктор: создать 4 точки маршрута, базу в центре, БПЛА в первой точке.

3. Слот tick():

```
QPointF target;
if (mode == MISSION) target = waypoints[wpIndex];
else if (mode == RTL) target = base;
else return; // LANDED
```

```
QPointF dir = target - uavPos;
double dist = dir.length();
if (dist > 3) {
    uavPos += dir / dist * 3;
    uav->setPos(uavPos);
}
```

```

} else {
    if (mode == MISSION) {
        wpIndex = (wpIndex + 1) % waypoints.size();
    } else if (mode == RTL) {
        mode = LANDED;
        uav->setBrush(Qt::red);
        log("Посадка завершена");
    }
}
}

```

4. Кнопка «Обрыв связи»:

```

mode = RTL;
uav->setBrush(Qt::yellow);
scene->addLine(uavPos.x(), uavPos.y(), base.x(), base.y(),
               QPen(Qt::yellow, 1, Qt::DashLine));
log("Потеря связи! Переход в режим RTL");
ui->labelStatus->setText("Статус: ВОЗВРАТ");

```

Контрольные вопросы:

№	Вопрос
1	Какие параметры полётного контроллера используются для расчёта траектории RTL?
2	Что произойдёт, если при RTL батарея разрядится ниже критического порога? Какой режим активируется?
3	В чём разница между RTL и Emergency Landing? Когда применяется каждый?

Вариант 9. Визуализация зоны покрытия радиосвязи

Задача: Пользователь указывает положение базовой станции (щёлочком по сцене) и её радиус (ползунком). Программа рисует круг зоны покрытия и размещает 5 БПЛА в случайных точках. Каждый БПЛА окрашивается зелёным, если он в зоне, красным — если вне зоны. Метка показывает число БПЛА в зоне.

Эскиз окна:

Зона покрытия радиосвязи	
QGraphicsView	Радиус: —[===]— 200 м
синий круг – зона покрытия	[Разместить БПЛА] [Сброс]
зелёные круги – в зоне	
красные – вне зоны	В зоне: 3 из 5
синий квадрат – базовая станция	

Виджеты Qt Designer:

objectName	Тип	Назначение
graphicsView	QGraphicsView	Сцена с зоной и БПЛА
sliderRadius	QSlider	Радиус зоны
btnPlace	QPushButton	Размещение БПЛА
btnReset	QPushButton	Сброс
labelInZone	QLabel	Число БПЛА в зоне

Алгоритм решения:**1. mainwindow.h:**

```
QGraphicsScene *scene;
QGraphicsEllipseItem *zone;
QGraphicsRectItem *base;
QList<QGraphicsEllipseItem*> drones;
QPointF basePos;
double radius;
```

2. Конструктор:

```
scene = new QGraphicsScene(0, 0, 600, 400, this);
ui->graphicsView->setScene(scene);
basePos = QPointF(300, 200);
radius = 150;
base = scene->addRect(basePos.x()-10, basePos.y()-10, 20, 20,
    QPen(Qt::blue), QBrush(Qt::blue));
```

3. Обработка клика по сцене (перемещение базы):

```
ui->graphicsView->setMouseTracking(true);
connect(this, &MainWindow::sceneClicked, this, [=](QPointF p){
    basePos = p;
    base->setPos(p.x()-10, p.y()-10);
    redrawZone();
    checkDrones();
});
```

4. Кнопка «Разместить БПЛА»:

```
for (auto d : drones) scene->removeItem(d);
drones.clear();
for (int i = 0; i < 5; ++i) {
    double x = rand() % 600;
    double y = rand() % 400;
    auto *d = scene->addEllipse(x-8, y-8, 16, 16);
    drones.append(d);
}
checkDrones();
```

5. Функция checkDrones():

```
int inZone = 0;
for (auto d : drones) {
```

```

QPointF dp = d->pos() + QPointF(8, 8);
double dist = QLineF(dp, basePos).length();
if (dist <= radius) {
    d->setBrush(Qt::green);
    inZone++;
} else {
    d->setBrush(Qt::red);
}
}
ui->labelInZone->setText(QString("В зоне: %1 из 5").arg(inZone));

```

Контрольные вопросы:

№	Вопрос
1	Как затухание радиосигнала зависит от расстояния? Какую модель используют для расчёта зоны покрытия (free-space path loss)?
2	Что такое диаграмма направленности антенны и как она влияет на форму зоны покрытия?
3	Как рельеф местности учитывается при расчёте зоны покрытия радиосвязи БПЛА?

Вариант 10. Мониторинг парка БПЛА: таблица состояния

Задача: В таблице отображаются 6 БПЛА с параметрами: ID, заряд, статус (Online/Offline), координаты. Каждые 2 секунды значения обновляются (имитация телеметрии). Если заряд < 20%, строка подсвечивается жёлтым; если Offline — красным. На сцене точки БПЛА отображаются цветами, соответствующими статусу.

Эскиз окна:

Мониторинг парка БПЛА				
	ID	Заряд	Статус	
	1	80%	On	
QGraphicsView	2	15%	On	(жёлтая)
6 точек – БПЛА	3	0%	Off	(красная)
зелёные – Online	4	60%	On	
жёлтые – <20%	5	45%	On	
красные – Offline	6	90%	On	
	Обновлено: ...			
	[Пауза] [Сброс]			

Виджеты Qt Designer:

objectName	Тип	Назначение
graphicsView	QGraphicsView	Карта с точками БПЛА
tableWidget	QTableWidget	Таблица состояния
btnPause	QPushButton	Пауза / возобновление
btnReset	QPushButton	Сброс
labelUpdated	QLabel	Время последнего обновления
timer	QTimer	Периодическое обновление

Алгоритм решения:**1. mainwindow.h:**

```
struct DroneInfo {
    int id;
    int battery;
    bool online;
    QPointF pos;
    QGraphicsEllipseItem *item;
};
QList<DroneInfo> fleet;
QTimer *timer;
```

2. Конструктор:

```
ui->tableWidget->setColumnCount(4);
ui->tableWidget->setHorizontalHeaderLabels({"ID", "Заряд", "Статус", "Координаты"});
ui->tableWidget->setRowCount(6);
```

```
for (int i = 0; i < 6; ++i) {
    DroneInfo d;
    d.id = i + 1;
    d.battery = 30 + rand() % 70;
    d.online = (rand() % 10 > 1);
    d.pos = QPointF(rand() % 600, rand() % 400);
    d.item = scene->addEllipse(d.pos.x()-6, d.pos.y()-6, 12, 12);
    fleet.append(d);
}
```

```
timer = new QTimer(this);
connect(timer, &QTimer::timeout, this, &MainWindow::updateFleet);
timer->start(2000);
updateFleet();
```

3. Функция updateFleet():

```
for (int i = 0; i < fleet.size(); ++i) {
    auto &d = fleet[i];
    if (d.online) {
        d.battery -= rand() % 5;
```

```

    if (d.battery <= 0) { d.battery = 0; d.online = false; }
    d.pos += QPointF(rand()%20-10, rand()%20-10);
}

ui->tableWidget->setItem(i, 0, new QTableWidgetItem(QString::number(d.id)));
ui->tableWidget->setItem(i, 1, new QTableWidgetItem(QString::number(d.battery) +
"%"));
ui->tableWidget->setItem(i, 2, new QTableWidgetItem(d.online ? "Online" : "Offline"));
ui->tableWidget->setItem(i, 3, new QTableWidgetItem(
    QString("%1, %2").arg(d.pos.x(), 0, 'f', 0).arg(d.pos.y(), 0, 'f', 0)));

QColor color;
if (!d.online) color = QColor(255, 200, 200);
else if (d.battery < 20) color = QColor(255, 255, 200);
else color = QColor(200, 255, 200);
for (int c = 0; c < 4; ++c)
    ui->tableWidget->item(i, c)->setBackground(color);

d.item->setPos(d.pos.x()-6, d.pos.y()-6);
if (!d.online) d.item->setBrush(Qt::red);
else if (d.battery < 20) d.item->setBrush(Qt::yellow);
else d.item->setBrush(Qt::green);
}
ui->labelUpdated->setText("Обновлено: " +
    QDateTime::currentDateTime().toString("hh:mm:ss"));

```

4. Кнопка «Пауза»:

```

if (timer->isActive()) { timer->stop(); ui->btnPause->setText("Продолжить"); }
else { timer->start(2000); ui->btnPause->setText("Пауза"); }

```

Контрольные вопросы:

№	Вопрос
1	Что такое телеметрия БПЛА и какие основные параметры передаются на наземную станцию?
2	Какие протоколы используются для передачи телеметрии (MAVLink, UAVCAN)? В чём их отличие?
3	Как обрабатывается потеря пакета телеметрии на наземной станции? Что показывает «возраст данных»?